W4 第一课 · RAG 五环节与检索实验

从「你亲手找资料」到「脚本自动找资料」—— 这一课要做的事，是把检索这一环从黑箱里拽出来

W4 · RAG 入门 | 配套脚本 tools/mini_rag/mini_rag.py | 实测语料 47,598 字 / 114 块 | 写入 2026-11-03

本课目录

1. 一句话说透 RAG —— 开卷考试这个比喻
2. 五个环节长什么样 —— 流水线图 + 硬件类比
3. 为什么这一课不装 Dify —— 你机器的实测数据
4. 三组对照实验（本周核心作业） —— 含三道题与参考答案
5. 跑一遍脚本 —— 读代码 4/12：四段数据流
6. 切分大小对比实验 —— 200 / 500 / 1000
7. 答错了到底是谁的错 —— 一张诊断树
8. 五个必踩的坑 —— 含本次实测撞到的两个
9. 能不能做多层 RAG —— 「一级二级三级」这个想法对不对
10. 验收自测 —— 四问，全部有标准答案

一、一句话说透 RAG

RAG 不是给模型加知识，是给它一份参考资料，让它开卷考试。

你现在用的模型，脑子里装的是全人类的公开语料，它不知道你公司的样机判据怎么定、不知道你主表为什么是 20 列。 你想让它在你的文档上答题，有两条路：

路子	做法	代价
微调（不改这周学）	把你的文档喂进去重新训练一遍参数	贵、慢、改一次要重训；而且模型会「记混」——它把文档压缩进参数，细节就丢了，还会一本正经地编
RAG（本周学）	文档不动，每次提问时先把相关段落**查出来**，连同问题一起塞给模型	文档随时改、随时生效；答案能标注出处；查不到就说查不到

> **记住这一句就够本课了：** RAG 里模型没变聪明，变的是它答题时手边多了几页复印件。 所以**答案准不准，八成取决于「那几页复印件找得对不对」**，不到两成取决于模型本身。 这也是为什么 W5 要

> 花一整周去优化「召回率」—— 找错了页，模型再强也只能照着错页答。

二、五个环节长什么样

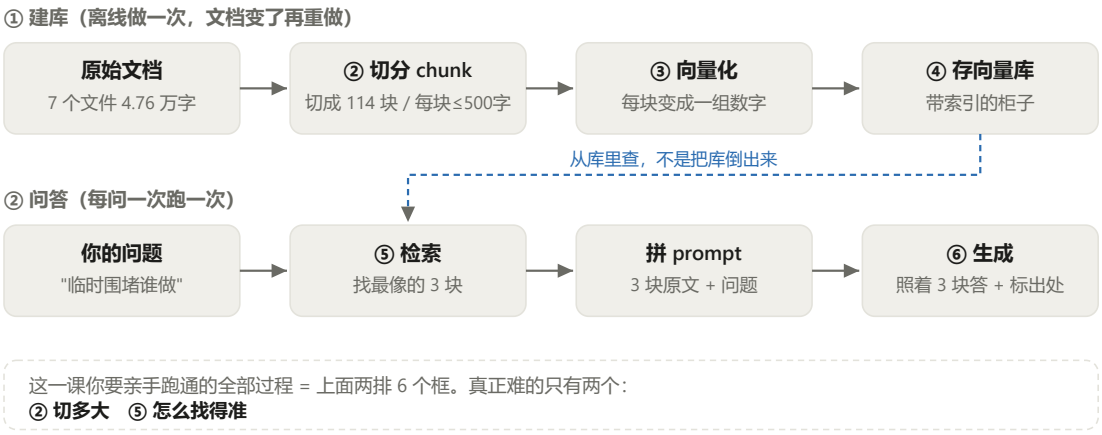


图 1 · RAG 六个环节。上半截建库只做一次，下半截每次提问都跑一遍

用你熟的场景翻译一遍

环节	RAG 里的动作	硬件语境里的对应物
1 语料	把你的一堆文档收进来、转成纯文字	把方案书、作业单、台账统统摊在桌上 —— 注意 PDF 扫描件要先 OCR，这一步最容易出脏数据
2 切分	长文档切成一小块一小块	把整本说明书拆成一页页 **SSOP 工位卡**：一张卡只说一件事，工人拿起来就能干活。切太碎则一句话没说完就断，切太大则一张卡混了三件事
3 向量化	每块算出一组数字（向量）	给每张卡打**特性指纹**，像器件丝印编码：同型号的指纹挨得近。这一步之后，"围堵"和"止血"即使字不同，指纹也可能很近
4 存库	把指纹和原文一起存进数据库	带坐标索引的**物料柜**：抽屉按指纹相似度排，抽哪一格不用翻全柜
5 检索	问题也算成指纹，找最像的 K 块	拿故障现象去柜子里**抽 3 张卡**，不是把整个柜子倒出来

6	生成	3 块原文 + 问题一起给模型	把 3 张卡的复印件夹在工单后面交给工程师：**照卡答，不许凭记忆**

> **一个数字先记住**：你这份语料切完是 **114 块**，检索只取 **3 块**。 也就是说，模型每次只看到全部资料的 **约 2.9%**（3×463÷47598 = 1,389/47,598）。 **剩下 97.1% 它一辈子也看不到。**这就是 RAG 全部威力和全部风险的来源 —— 找对了是神力，找错了是灾难。

三、为什么这一课不装 Dify

计划文件里 W4 任务三写的是"用 Dify 或 LlamaIndex 搭第一个知识库"。我建议这一课先不装，理由是实测数据：

项目	实测值	结论
本机总内存	13.8 GB	—
当前空闲内存	**2.5 GB**	Dify 全栈（约 15 个容器）要 4–6 GB，**现在装必然吃 swap**
Docker	未安装	装 Docker Desktop 本身再吃 1–2 GB
Ollama（本地模型）	未安装	7B 量化模型要 5 GB 以上，更不可能

> **所以本周的路线：先看清，再上工具。**
> 用**本地 Python 脚本 + 你已有的 DeepSeek key** 跑通五环节 —— 内存只要几十 MB，全程看得见每一步的输入输出。 等你亲手见过"切分—索引—检索"长什么样，再去 Dify 里点按钮，才知道那些按钮在替你干什么。 **反过来先学 Dify，你会变成"会拖节点但不知道节点在干嘛"的人**——这正是招聘里说的"vibe coding 但不懂架构"。

Dify 什么时候上：W5 建评测集的时候。那时你需要频繁改配置、跑批量对比，界面的效率才划算。届时先关掉一半浏览器标签把内存清到 6 GB 以上。

四、三组对照实验（本周核心作业）

这一步**先不用脚本**，让你亲手当一次检索器。做完你会得到一个别人讲不明白的体感： **同一份资料、同一个模型，只是"给它的资料范围"不同，答案质量能差出两个档次。**

语料：你自己写的 7 份文档

```
# 实测规模（2026-11-03）
新国标大点位控制器_缺陷闭环系统_方案建议书_v1.4.html     16,204 字
缺陷闭环系统_字段配置器.html                               1,977 字
缺陷闭环系统_智能表格建表作业单_v2.html                   12,944 字
缺陷闭环系统_字段填写顺序与依赖图.html                    2,779 字
缺陷闭环系统_45字段全流程图.html                            983 字
任务拆分_交接卡.html                                      7,525 字
缺陷闭环系统_字段总台账.csv                               5,186 字
————————————————————————————————————————————
合计 7 个文件 / 47,598 字 → 切成 114 块 / 平均 463 字一块
```

> **先纠正我上一轮的一个判断。**我原估这 7 份"约 25 万字"，实测只有 **4.76 万字。** 所以实验设计改了：**不拿"塞不下"当卖点** —— 4.76 万字约 3.2 万 token，豆包 25.6 万窗口塞得进，才占 12%。 真正的痛点不是装不下，是**成本和"塞进去了它也找不着"。**下面的实验按这个来。

三道题（答案只在上面这堆文档里，你熟，能秒判对错）

Q1 · 跨文档对照　「紧急围堵措施」和「围堵措施（临时）」，分别是哪个阶段填、谁填、什么时候填?

答案　紧急围堵 = **质量部**，受理当天就做，S/A 级才触发（C 组"定级与围堵"字段）；围堵措施（临时）= **研发**，根因分析之后做（E 组"对策"字段）。两者信息前提不同，前后差 3~5 个工作日，所以不能合并。**出处**：依赖图 第五节「五个最容易填错的地方」第 4 条 / 方案书 v1.4 第三节「② 定级围堵」

Q2 · 数字对账　主表建了几列？47 个字段里必填几个？还有多少列延后?

答案　主表 **20 列** = 用到其中 **16 个字段** + 4 个系统/状态列（创建人、最后更新时间、当前状态、当前责任人）；47 个字段里**必填 10 个**。延后数量见下方冲突提示。

Q3 · 判据防误导　样机判据是不是「T3 ≤ T2」?

答案　**不是，不能一刀切。**要拆成 T3a（方案锁定，必然 ≤ T2）和 T3b（物理齐套：改固件应 ≤ T2，**换料必然 > T2**）。拿"T3 ≤ T2"去考核等于考核一件物理上做不到的事。**出处**：方案书 v1.4 第二节「为什么 T3 要拆成 T3a / T3b」/ 交接卡「已经定死、不要重新讨论的决策」里的"样机判据"那一条

> **Q2 里我埋了一个真实的坑，你自己核一下。**
> 建表作业单 v2 原文写「主表 20 列 = 用到其中 16 个字段 + 4 个系统/状态列……**其余 32 个字段延后**」。
> 但算一遍：`47 − 16 = 31`。差 1。 ❓

> 两种可能：① "32" 是笔误，应为 31；② 口径里把某个系统列也算成了一个待建字段。 **这个我不替你定** —— 你确认后我改文档。这也正好是本周的活教材：**RAG 会把文档里原有的错误，原样放大成"权威答案"。**

三组怎么做（每组 10 分钟）

组	你做什么	预期看到什么
A 全塞	把 7 份文档的全文（或尽量多）一次性贴进对话框，然后问 Q1	答得大致对，但会开始"综合"和"脑补"，出处说不清。
B 人工检索	只贴**下面那个灰框里的原文**（依赖图「五个最容易填错的地方」第 4 条），再问 Q1	答得干净、准确、能指出依据。这就是 RAG 的终态。
C 错检索	只贴**方案书里"定级围堵"阶段**那段（只讲紧急围堵、没提临时围堵），再问 Q1	**模型会理直气壮地答错**：它会说围堵就是质量部受理当天做。它没撒谎 —— 它只看到那一页。

B 组要贴的原文，直接从这里复制，不用去翻文件：

```
4  把「围堵」和「临时对策」当成一件事
C 组「紧急围堵措施」（质量部，受理当天，S/A 级才填）＝先把客户的货挡住，在根因还不知道的时候做。
E 组「围堵措施（临时）」（研发，根因分析之后）＝知道了原因之后采取的短期手段，是和永久对策配套的。
两者信息前提不同，所以不能合并，前后大概差 3~5 个工作日。
```

出处：`缺陷闭环系统_字段填写顺序与依赖图.html` → 第五节「五个最容易填错的地方」第 4 条（源文件第 239 行起）。
顺带说清「第 77 块」是什么：那是脚本把文档切成 114 块后给每块编的内部序号，存在 `_out/index.json` 里，**原文档页面上看不到。** 想按编号查原文，用 `python mini_rag.py --show 77`。上一版教程只给了编号没给原文，是我表述不清。

C 组要贴的原文（故意缺一半的那一页），也直接复制：

```
② 定级围堵
严重程度定级；先止血：库存隔离 / 在制品拦截 / 客户端更换 / 固件回滚；
并给出上报 / 召回处置的初判
```

```
质量经理 + 研发主管会签（S/A 级须 24h 内）
出口标准：围堵动作已执行并有证据；影响范围已锁定；是否上报 / 召回已由人决策并记录
```

出处：`新国标大点位控制器_缺陷闭环系统_方案建议书_v1.4.html` → 第三节「生命周期五阶段与出口标准」表格里「② 定级围堵」那一行（脚本里是第 5 块）。
为什么这页会坑人：它把围堵写得完整又权威 —— 有动作、有责任人、有出口标准，唯独没说"这只是紧急围堵，临时围堵是杀完根因之后研发做的"。 模型拿到这一页，没有任何理由怀疑自己答错了。

> **C 组这一下必须做。**它让你亲身看到：**模型不会说"资料不足"，它会用手里那点资料编出一个完整答案。** 这就是 RAG 系统翻车的 90% 原因，也是 W5 要建评测集的理由。

顺手记录（这就是作业产出）

组	答案对不对	有没有编	能不能说出依据
A 全塞			
B 人工检索			
C 错检索			

五、跑一遍脚本（读代码 4/12 的实物）

脚本放在 `E:\workbuddy workspace\tools\mini_rag\`，两个文件：`mini_rag.py`（275 行，零第三方依赖）和 `run.bat`（一键菜单，双击就能用）。

> **先纠正上一版教程的一个疏漏：我只写了「在目录下执行 python mini_rag.py」，没写「在哪个窗口、怎么进到那个目录」。**
> 这一步对不熟悉命令行的人是硬门槛 —— 双击 .py 文件窗口一闪就没，从「开始菜单」打开 cmd 又停在 C:\Users\lenovo，敲什么都是「找不到文件」。 所以下面按**「双击」「敲命令」两条路**分开写，你选一条即可。

方式一：双击 run.bat（推荐，不用记任何命令）

1. 按 **Win + E** 打开文件资源管理器，**在地址栏里直接粘贴**这一行再回车：
   `E:\workbuddy workspace\tools\mini_rag`

比一层层点进去快，也不会点错目录。

2. 窗口里应该看到三样：`mini_rag.py`、`run.bat`、`_out`（文件夹）。
   双击 `run.bat`（图标带齿轮/窗口那个）。
3. 弹出一个黑窗口，标题是「mini_rag 最小检索实验台」，下面是 7 行菜单。
   可能弹一次「Windows 已保护你的电脑」→ 点「更多信息」→「仍要运行」（bat 是本机脚本，不是下载的安装包）。
4. 键盘输入 **2**，回车。**这就是第一件事**：跑 Q1，只看检索、不调模型，所以不需要任何 API key。
5. 屏幕刷出五段，末尾一行「按任意键回菜单」。**敲一下回车**回菜单，输 **0** 回车退出。

序号	跑什么	什么时候用
1	自己输入一个问题，跑完整五段	最后一步会调 DeepSeek 模型。**没配 key 会报错，先跳过**，等第六节看完再说
2	Q1「临时围堵谁做」，只看第 4 段	**先跑这个。**免 key，出的结果就是第四节 B 组要的原文
3	Q2「主表建了几列」，多捞 2 块（--topk 5）	看「多给几块」会不会反而带进更多噪声
4 / 5	同一个问题，切 200 字 / 1000 字	第六节的切分对比实验，两次跑完自己填表
6	按编号看某一块原文（默认 77）	第 4 段说「77 块」时，用它调出全文核对，别靠预览下结论
0	退出	—

方式二：自己敲命令（想改参数时用）

1. 同样先打开那个文件夹窗口（地址栏粘 `E:\workbuddy workspace\tools\mini_rag` 回车）。
2. **在地址栏把路径全删掉，输入 `cmd`，回车。**黑窗口直接开在这个目录里 —— 这是 Windows 自带的技巧，不用装终端、不用配环境。
3. 确认没进错目录：敲 `dir` 回车，列表里能看到 `mini_rag.py` 就对了。
   黑窗口提示符前面显示的就是当前目录。这一步的目的就是「先确认站对地方，再敲命令」。
4. 粘贴下面任意一行回车：

```
python mini_rag.py "临时围堵措施由谁在什么时候做"           # 五段全跑，最后调模型
python mini_rag.py "临时围堵措施由谁在什么时候做" --no-llm  # 只检索，不调模型（免 key）
python mini_rag.py "主表建了几列" --topk 5                  # 多捞几块
```

```
python mini_rag.py "样机判据" --chunk 200 --no-llm          # 切分实验
python mini_rag.py --show 77                                # 按编号看某块全文
```

你机器上 `python` 已经在 PATH 里（3.13.14），所以上面这样写就能跑。**万一换了电脑报「'python' 不是内部或外部命令」**，把 `python` 换成完整路径：

```
"C:\Users\lenovo\.workbuddy\binaries\python\envs\default\Scripts\python.exe" mini_rag.py "你的问题"
```

报错对照表：照症状找原因

屏幕上出现	真实原因	怎么办
`'python' 不是内部或外部命令`	python 不在 PATH，或这台机器没装	用上面的完整路径；或直接双击 `run.bat`，它会自己找 python
`can't open file 'mini_rag.py': [Errno 2]`	**当前目录不对**（最常见）	敲 `dir` 看有没有这个文件；没有就先 `cd /d "E:\workbuddy workspace\tools\mini_rag"`
中文显示成 `锟斤拷` 之类	窗口代码页不是 936	双击 `run.bat`（已内置处理）；或敲 `chcp 936` 再跑
`ModuleNotFoundError`	不该出现（本脚本零依赖）	说明 `python` 指向了别的环境，用完整路径跑
窗口一闪就关	你双击的是 `.py` 而不是 `.bat`	双击 `run.bat`；或按方式二先开 cmd

跑出来应该长什么样（我 11-03 的实测输出，节选）

选 **2** 之后，依次刷出五段。**每一段的标题就是那一环的名字**，这就是图 1 那六个框的实际执行：

```
===== 第 1 段 读文件 =====
  [读取] 新国标大点位控制器_缺陷闭环系统_方案建议书_v1.4.html      16204 字
  [读取] 缺陷闭环系统_字段配置器.html                                1977 字
  ...（共 7 个文件）
  合计 7 个文件，47598 字

===== 第 2 段 切分（每块 500 字上限，重叠 50）=====
  切成 114 块，平均 463 字/块
```

```
===== 第 3 段 建索引（双字切片 + TF-IDF，存成 json 供查看）=====
  词表 8745 个，索引已写入 _out/index.json（可以打开看看长什么样）

===== 第 4 段 检索（问题：临时围堵措施由谁在什么时候做）=====
  [1] 0.190  《缺陷闭环系统_字段总台账.csv》第 107 块
  [2] 0.131  《缺陷闭环系统_字段填写顺序与依赖图.html》第 77 块
  [3] 0.113  《缺陷闭环系统_智能表格建表作业单_v2.html》第 65 块

===== 第 5 段 拼 prompt =====
  prompt 共 1663 字，已写入 _out/prompt.txt（这就是交给模型的全部内容）
```

> **第 4 段这三行，是这一课最值钱的东西，别扫一眼就过。**
> **排第 1 的是一行 CSV，跟问题毫无关系**（它是字段说明里"相似历史缺陷编号"那一行的副本）；**真正能回答的依赖图第 77 块只排第 2。**
> 为什么？因为你的问题里「围堵」「临时」「措施」这些词，在 CSV 的字段说明里出现得更密。**TF-IDF 只数字面，不懂你在问什么。**
> 这就是「检索到了，但没排到第一」—— 工程上靠 `--topk 3` 兜住，靠 rerank 排序，靠评测集验证。**你现在看到的是问题的原型，不是脚本写坏了。**
> 想亲手确认第 2 名那段就是答案：菜单选 **6**，输 `77`，全文直接打出来，和第四节 B 组那张卡对照。

顺带一句： `run.bat` **是纯文本**，右键「编辑」就能看到它其实只有十几行、每行就是一条命令。这也是读代码 —— 而且是你唯一能 100% 看懂的代码。

四段数据流：这就是 4/12 要求你指出的四件事

问题	在哪（行号）	代码在做什么
① 读了哪些文件	`配置区 24–32` `load_files 54–66`	把 7 个文件名写死在一个列表里；`strip_html()` 剥掉 HTML 标签只留人话，CSV 直接读
② 切成什么	`chunk_docs 72–93`	先按段落切，再贪心累加到 ≤500 字；相邻块重叠 50 字，防止一句话被拦腰截断

③ 存到哪	`build_index 114–132`
	落盘在
	`_out/index.json`
		给每块算"词权重向量"，连同原文一起写进 json。**打开这个文件看一眼**，你就知道"向量库"里到底存了什么

④ 怎么查出来	`search 137-150`	问题也切成同样的词 → 算权重 → 跟 114 块逐一算余弦相似度 → 排序取前 K

> **一个必须现在讲清的差别：这个脚本的"向量化"是穷人版。**
> 它用**中文双字切片 + TF-IDF**（"紧急围堵"→ 紧急/急围/围堵），只能匹配**字面**。 真 embedding 是模型算出来的几百维向量，能匹配**意思**：你问"怎么止血"，它能捞到写着"围堵"的那块。
> 本脚本做不到这一点。**这不是缺陷，是设计** —— 让你先看清"检索"这个骨架，再换算法。 换真 embedding 只需改第 3 段和第 4 段（`build_index` / `search`），其余四段一行不动。
>
> **实测两例，你复现时注意区别：**
> · 问「产品出问题后怎么先止血」→ **能捞到**，因为文档原文里本来就有"先止血"三个字。这是字面命中，不是理解了意思。
> · 问「供应商来料有问题怎么开单」→ **翻车**，Top-1 落到"加列可以，改类型不行"那块。因为文档里没有"来料""开单"这种说法。
> **字面命中 ≠ 语义命中。**分不清这两者，你就会误以为检索器"能用"。

六、切分大小对比实验

计划文件 W4 第四条要求"亲手对比 200/500/1000，别信别人的结论"。用同一个问题跑三次，只改 `--chunk`：

```
python mini_rag.py "样机判据是 T3 小于等于 T2 吗" --chunk 200
python mini_rag.py "样机判据是 T3 小于等于 T2 吗" --chunk 500
python mini_rag.py "样机判据是 T3 小于等于 T2 吗" --chunk 1000
```

切分	块数	Top1 捞到什么	判断
200 字	362 块（平均 180 字）	交接卡「样机判据分两档：T3a…」——**只给结论，不给理由**	命中快、解释少；问"为什么"就会答不出来
500 字	114 块（平均 463 字）	方案书那块——**完整解释了"为什么不能一刀切"**	本次基线，解释最完整
1000 字	53 块（平均 938 字）	方案书判据表那段，但 **Top2/Top3 落到完全无关的块**	大块混主题，相似度被平均稀释，垃圾进榜

更正一处：我上一版把这个实验看错了，写进教程的是错的，现在改过来。
我原先写"Top1 那块话说一半就断了、结论被切到下一块"。**错了。**我是拿脚本打印的 **110 字预览**当成了块全文 —— 用 `--show 11` 调出全文才发现，方案书那块 484 字，从"为什么 T3 要拆成 T3a / T3b"一直写到三种整改类型的判据表，**答案完整。**

这个乌龙本身值得记：预览截断 ≠ 块截断。以后排查"是没检索到还是检索到了没用上"，**必须调全文，不能看预览下结论。**我这次就栽在这儿。

那到底有没有被切伤的地方？有，但在别处：同一块的结尾，判据表第三行「改工艺 / 改检验 / 更新文件」**只写了半行**就断了 —— `--show 11` 的最后一个词是"不涉及样机状态"，后面没了。它不影响本题（答案在前半段），但换一个问法就会缺一块。
所以准确的说法是：切伤要看切在哪一句，不是切了就有害。这就是为什么第 6 节这张表必须你自己跑 —— 你问的问题不同，结论可能相反。

七、答错了到底是谁的错

计划文件 W4 第五条要求你分清"没检索到"和"检索到了没用上"。这两种病治法完全相反，判断方法只有一步：**先看检索结果里有没有正确答案。**



图 2 · 先判病再开药。搞反了就会陷入"疯狂改 prompt 但检索一直是错的"的死循环

脚本里已经给你留了这个钩子。第 4 段会把每一块的原文打出来 —— **不要只看最终答案，先看那 3 段原文。** 养成这个习惯，你排查 RAG 问题的速度会比别人快一个数量级。

让模型"先引用再作答"

计划文件 W4 第六条要求试这个官方技巧。脚本的 prompt 模板（`第 153–177 行`）已经按这个写法定好了三条规则：

```
# PROMPT_TMPL 里的三条硬规矩
1. 每条结论后面用［编号］标出来自哪一段资料，禁止凭记忆补充。
2. 资料里没有的，直接回答"资料中未提及"，不要猜。
3. 如果不同资料说法不一致，把两种说法都列出来并指出冲突。
```

第 3 条不是凑数 —— **Q2 那个 31/32 的数字冲突，就是靠这条规矩才被暴露出来的。**没有它，模型会挑一个数字直接说，你永远不知道文档里有两个版本。

八、五个必踩的坑

坑	现象	怎么认 / 怎么修
① 字面匹配抓不到同义词 本次已撞到	问"供应商来料有问题怎么开单"，Top-1 居然落到"加列可以，改类型不行"那一块，完全不相干	本脚本必然如此（TF-IDF 只认字面，"来料/开单"在文档里不这么写）。修法：换真 embedding。这也是你以后判断"该不该上向量库"的依据
② 切块把结论切断 本次已撞到	方案书那块（`--show 11`）结尾停在"改工艺 / 改检验 / 更新文件…不涉及样机状态"，判据表第三行的结论没了	本例不影响原问题（答案在前半段），换问法就会缺。修法：加大 `OVERLAP`，或按标题层级切而不是按字数切
③ 切太小 / 切太大	太小：一块半句话，模型看不懂上下文 太大：一块混三件事，相似度被稀释	用第 6 节的三次对比定，别抄别人的 500
④ 只取 Top-3 太吝啬	答案在第 5 块，但只喂了前 3 块	`--topk 5` 试一次；代价是 prompt 变长、成本上升 —— 这就是要用 W3 成本表算的地方
⑤ 文档本身就是错的 本次已撞到	31 还是 32，文档自己没对齐	RAG 会把错误原样放大成权威答案。修法是改文档，不是改模型 —— 这也是为什么你的缺陷台账必须只有一个"唯一定义源"

九、能不能做多层 RAG（11-04 追加）

原提问：素材很大时（例如《PMBOK 指南》17 万字），是不是可以做一级 RAG、二级 RAG、三级 RAG 这样往下叠，就能把内容检索得更充分？

> **结论：方向半对，做法反了。**
> 「分层检索」是真的、也真的在用（有正式名字），但**你设想的「一级筛完给二级、二级筛完给三级」恰好是论文里被否掉的那种走法**； 而且 17 万字**远不到需要它的量级。**你现在的问题不是「层不够多」，是「第一层就常排错」。

先承认：这门技术存在，只是不叫「一级二级三级」

正式名字	它怎么分层	什么时候用
父子文档检索 `parent-child` / small-to-big	小块拿去检索，命中后把它的**父块（整节）**喂给模型	生产上最常用。**这正是「两层」的正确形态：细找、粗给**
RAPTOR 递归摘要树	把块聚类 → 让模型写摘要 → 摘要再聚类再摘要，形成多层树	问题需要**全局综合**时（"这本书主线是什么"），单层检索根本答不出
查询路由 `routing`	先判断该查哪一章 / 哪张表，再进那一块里检索	语料有清晰目录结构时，最便宜的一层

注意共同点：这三个的分层都是用来「改变粒度」的，没有一个用来「逐级淘汰」。

为什么「逐级往下筛」会亏

不用讲概率，一条逻辑就够：**第二级只能在第一级交上来的候选里挑。第一级没留住的那块，后面永远拿不回来。** 所以串联结构里，**最终召回率的上限被第一级锁死**，叠得越多，能答对的机会只会更少、不会更多。

如果每级各自有 90% 的把握留住正确答案，串起来是这样：

每级留住正确答案	串 2 级	串 3 级	串 4 级
95%	90%	86%	81%
90%	81%	73%	66%
80%	64%	51%	**41%**

真实系统里各级并非彼此独立（第一级给的范围越大，后面越容易救回来），实际衰减比这张表缓一些。 但方向不会变：**每多一级，就多一个「必须对」的关口。**

> **RAPTOR 原论文自己做过这两种走法的对比，结论对你不利：**
> · **Tree Traversal（逐层下降）**：从根往下，每层只进选中节点的子树 —— **一旦顶层摘要和问题措辞对不上（词汇不匹配）或摘要过度抽象，整棵子树就被永久排除。**
> · **Collapsed Tree（压平）**：把叶块和各级摘要**倒进同一个池子**，一次相似度打分，跨层取前 k —— 论文实测**一致优于**逐层下降。
> 换句话说：树可以建，但**检索时不能顺着树往下爬，要横着捞。**

那 17 万字到底要不要分层

先算三个数	结果
17 万字大概多少 token	按你 7 份文档实测的 **0.67 token/字**折算 ≈ **11.4 万 token**。豆包 25.6 万窗口**塞得下**
切成 500 字一块是多少块	约 **340 块**
取 Top-5 喂给模型是多少	2500 字 ≈ **全文的 1.5%**

所以 17 万字的真正难点不是「装不下」，是**「怎么让对的 5 块排进前 5」**。这件事跟层数无关，跟排序质量有关。

> **判断口诀：问「在哪」用一层，问「整体」才要树。**
> · PMBOK 的题九成是「某过程的输入输出是什么」「变更要走哪几步」—— **定位型**，一层足够，叠层只有坏处。
> · 只有「十个知识领域之间怎么耦合」「这本书的管理思想怎么演变」这类**全局综合题**，单层真的捞不到，那时树才有意义。
> 按这个标准，你手上「缺陷闭环系统」的提问 95% 是定位型（某字段谁填、某状态能不能跳）。**你不需要树。**

想提精度，按这个顺序做（性价比从高到低）

手段	怎么做	为什么排这个位置
① 结构过滤（元数据）	先按章节 / 阶段把范围缩到一章，再在章内检索	**成本接近零。**PMBOK 的目录、你文档的 8 阶段 47 字段，本身就是现成的过滤器。这一层不叫 RAG，叫「先别乱找」

② 混合检索	BM25（字面）+ 向量（语义）两路并取	专有名词（SCAR、ECN、T3b）靠 BM25 才稳；同义词靠向量。**第 8 节那个「来料/开单」翻车，就是只有字面这一路**
③ rerank 重排	先取前 50 块，再用重排模型挑出前 5	**提升最大的一步。**你现在 Top-1 常是垃圾、正确块排第 2，就是缺这一下
④ 父块回填	小块检索，命中后把整节喂进去	解决「块切断了结论」（第 8 节坑②）。**这才是两层：不是筛两次，是细找粗给**

真要建树，记住两条

一、检索时压平，不要往下爬（上面 RAPTOR 那段）。**二、先算代价：** 建树要在入库阶段额外让模型写摘要，约多花 **0.3 ~ 0.5 倍语料**的 token；索引规模膨胀到 **1.3 ~ 1.5 倍**；**文档一改，上层摘要就失效。** （数字来自 RAPTOR 论文及其工程解读，不同来源口径略有差异，量级可信、精确值别当定论 ❓）

最后一条对你最致命：你的方案书还在 v1.3 → v1.4.2 迭代，字段口径还在改 —— **每改一次就要重建上层摘要。** 树适合稳定的语料，不适合天天改的活文档。**你现在这套文档，恰恰是后者。**

> **一句话带走**：多层 RAG 不是「检索得更充分」，是「用不同粒度回答不同层级的问题」。
> 你现在缺的是**排序精度**，不是**层级数量**。所以 W5 该做的是评测集 + rerank，**不是往上叠层。**
> 来源：RAPTOR 原论文（Sarthi 等，ICLR 2024）及两篇工程解读；「压平优于逐层下降」为论文结论，「召回提升 20%–35%、索引膨胀 1.3–1.5 倍」为二手汇总口径。

十、验收自测（读代码 4/12）

四个问题，都能在代码里指出位置才算过。**不需要会写，只需要能指出。**

问题	标准答案
① 它读哪些文件?	`配置区 24–32 行` 的 `CORPUS` 列表，7 个文件名，都拼在 `BASE` 目录下
② 切成什么?	`chunk_docs`（72–93 行）：按段落切 → 累加到 ≤500 字 → 相邻块重叠 50 字
③ 存到哪?	`_out/index.json`，里面是 `{chunk_size, chunks[{id,src,text}], idf_top}`。**打开看一遍**

④ 怎么查出来?	`search`（137–150 行）：问题同样切词 → TF-IDF 权重 → 与每块算余弦 → 排序取前 K

> **加一道更硬的（能答出这题，说明你真读懂了）：**
> 「想换用真 embedding，要改哪几行？为什么其余部分不用动？」
> **答**：改第 3 段的 `build_index`（114–132）和第 4 段的 `search`（137–150），把"双字切片 + TF-IDF"换成"调 embedding 接口拿到向量 + 算余弦"。 其余四段不用动 —— 因为**切分、存储、拼 prompt 这三件事跟用什么算法算向量完全无关。** 这个"接口不变、内部可换"的分层，就是你能看懂任何 RAG 框架（LlamaIndex / Dify）的底层原因。

十一、本课产出清单

产出	对应计划任务
7 份文档切成的 114 块 + `_out/index.json`	W4-① 搞懂五环节
三组对照记录表（A 全塞 / B 人工检索 / C 错检索）	W4-⑤ 分清两种病
切分 200/500/1000 三次对比记录	W4-④ 亲手测切分
能指出四段数据流的行号	W4-⑦ 读代码 4/12

> **还有两件 W3 遗留，建议在这周顺手补掉（各 15 分钟）：**
> ① **读代码 3/12 没落地**：真改一次 key 跑通一次 API 调用。你的 DeepSeek key 在 Galapagos 里能拿到，改 `call_llm()` 的环境变量即可 —— **不做这步，W5 看报错会卡在第一行。**
> ② **「三个自动化」大概率只做了教程**：挑最简单的"日报素材自动汇总"实跑一次，留截图。宁可记录"只跑了 1 个"，也别写"3 个完成"。

下一课（W4 后半）：把穷人版检索换成真 embedding，然后把语料扩到 20-50 篇，看块数从 114 涨到多少、Top-3 命中率怎么变。W5 再回来建 50 条评测集 —— 那才是"专业"和"玩票"的分界。

本课说明

· 本文件是本机文件，双击即可打开，**不联网、不依赖任何平台、不上传任何数据。**

· 文中所有规模数字（47,598 字 / 114 块 / 463 字/块）均为 2026-11-03 在本机实跑所得，非估算。

· 内存数据（总 13.8 GB / 空闲 2.5 GB / Docker 与 Ollama 均未安装）为 2026-11-03 21:10 实测。

· token 换算按中文 1.5 字/token 粗算，实际以 API 返回的 `usage` 字段为准。

· 「其余 32 个字段延后」与 `47 - 16 = 31` 的差 1，标 ❓ 待嘉哥确认，不擅自改文档。

· **建议归档**：本文件请与 W3 其余教程放在同一目录，别再丢了 —— W3 那 6 份教程目前只剩一份验收记录在盘上。

· 巴蒂 for 嘉哥 · 2026-11-03